

PyDataFlow — Product Overview

What Is It?

PyDataFlow is a **desktop visual data pipeline builder** built in Python. It presents a node-based canvas where a user drags tools from a palette, connects them with wires, configures their parameters in a side panel, and runs the resulting pipeline to process tabular data. Every step is powered by **pandas** under the hood, and the application can export the entire pipeline as a clean, standalone Python script.

The application runs entirely offline on the user's machine. There is no server, no cloud dependency, no licensing server, and no data ever leaves the local environment.

Core Interaction Model

Area	Description
Palette (left panel)	Categorised list of 22 tools. Drag a tool onto the canvas to add it.
Canvas (centre)	Infinite zoomable/pannable workspace. Nodes are positioned freely. Edges (wires) connect output ports to input ports with animated bezier curves.
Properties panel (right)	Context-sensitive configuration for the selected node. All column selectors are populated from live upstream data — no typing column names by hand.
Log tab (bottom)	Execution messages, row counts, timing.
Preview tab (bottom)	Scrollable table view of the last executed node's output.

Running a pipeline: Press "Run Flow" — the engine resolves a topological sort of all connected nodes and executes them in dependency order. Results flow downstream automatically. Individual nodes can be disabled (greyed out and skipped) without deleting them.

Saving / Loading: Pipelines are saved as JSON files (`.flow`). The format captures every node's position, type, and configured parameters.

Code Export: "Export Python" generates a self-contained `.py` file that reproduces the entire pipeline using `pandas` — no dependency on PyDataFlow at runtime.

Tool Catalogue

Input / Output (5 tools)

Tool	Description
Import CSV	Reads a CSV, TSV, pipe-delimited, or custom-delimiter flat file. Options: delimiter selection, custom delimiter, skip blank lines.
Import Excel	Reads an <code>.xlsx</code> / <code>.xls</code> / <code>.xlsm</code> file via <code>openpyxl</code> . Configurable sheet name or index.
Show Table	Pass-through node that pops open a scrollable data viewer during execution. Configurable row limit (default 500).
Export CSV	Writes the input DataFrame to a CSV file at a specified path. Option to include the pandas index.
Export Excel	Writes to an <code>.xlsx</code> file with a configurable sheet name.

Preparation (9 tools)

Tool	Description
Select Columns	Multi-select column picker to keep only the chosen columns. All available upstream columns appear as checkboxes. Column inference is used to drive downstream dropdowns.
Filter Rows	Structured condition builder. Each condition row has: a column dropdown (populated from upstream), an operator dropdown (13 operators — see below), and a value field . Multiple conditions combined with AND or OR.
Sort	Multi-column sort. Each sort key has a column dropdown and ascending/descending toggle. Unlimited sort keys.
Head / Tail	Keeps the first N or last N rows. Useful for sampling or quick inspection.
Rename Columns	

Tool	Description
	Dynamic row list: each row has an existing-column dropdown and a new-name text field. Add as many rename pairs as needed.
Edit Columns	Changes the data type of selected columns. Supported conversions: string, integer, float, datetime, boolean, category.
Add Columns	Adds one or more new computed columns. Each row specifies a column name and a formula expression. Includes a built-in formula reference guide covering numeric, text, date, conditional, and cast operations.
Cleansing	Batch data quality operations: trim whitespace, normalise to lowercase/uppercase/title case, replace empty strings with NaN, drop fully null rows, drop duplicate rows, fill NaN with a specified value.
Record ID	Appends a sequential integer ID column. Configurable column name and starting value.

Filter operators (13):

Operator	Behaviour
<code>= equals</code>	Exact match (string or numeric)
<code>≠ not equals</code>	Exclusion
<code>> greater than</code>	Numeric or date comparison
<code>≥ greater or equal</code>	Numeric or date comparison
<code>< less than</code>	Numeric or date comparison
<code>≤ less or equal</code>	Numeric or date comparison
<code>contains</code>	String substring check
<code>starts with</code>	String prefix check
<code>ends with</code>	String suffix check
<code>is null</code>	NaN / None check
<code>is not null</code>	Non-null check
<code>in list (a,b,c)</code>	Value in a comma-separated set
<code>between lo,hi</code>	Inclusive range check

Join / Reconcile (3 tools)

Tool	Description
Merge / Join	Joins two datasets. Join types: inner, left, right, outer. Key pairs are configured with separate left-column and right-column dropdowns (each populated from the respective upstream table). Multiple key pairs are supported; when more than one is defined, ordinal labels (1), (2) appear. Produces three output ports: <code>joined</code> , <code>left_unmatched</code> , <code>right_unmatched</code> — all three wirable independently.
Union	Stacks two datasets vertically. Match mode: by column name or by position. Optional <code>_source</code> column to tag which side each row came from. Shows a column preview from each input side in the properties panel.
Unique / Duplicate	Splits a dataset into two output streams: rows that are unique on a set of key columns, and rows that are duplicates. Keep policy: first occurrence, last occurrence, or flag all duplicates. Option to add a <code>_dup_group</code> integer to identify duplicate clusters.

Transform / Summarize (4 tools)

Tool	Description
Summarize	Runs <code>pandas.DataFrame.describe()</code> . Scope options: all columns, numeric only, or a specific column selection. Optional percentile inclusion. Optional transposition of the result.
Group By	Groups on one or more key columns (multi-select dropdown). Aggregation rows are configured individually: each row picks a column, an aggregation function (sum, mean, count, min, max, std, median, first, last), and an optional output alias.
Pivot	Reshapes long-format data to wide format using <code>pd.pivot_table</code> . Configurable index field, column field, values field, and aggregation function.
Unpivot	Reshapes wide-format data to long format using <code>pd.melt</code> . Configurable ID columns, value columns, and output variable/value column names.

Documentation (2 tools)

Tool	Description
Comment	A free-floating sticky note on the canvas. Title + body text. Does not participate in data flow.
Container	A resizable, collapsible grouping rectangle. Drag nodes into it to logically group a section of the pipeline. When collapsed, all contained nodes are hidden and their external edges are re-routed through the container boundary. Configurable title, description, and fill colour.

Column Intelligence

PyDataFlow traces the data flow graph backwards from each node to determine which columns are available at each input port. This means:

- Column dropdowns in every tool (filter, rename, group by, join keys, etc.) are **automatically populated** when upstream nodes have been executed or have a known schema (e.g. a configured CSV import).
- For multi-input tools (Merge/Join, Union), **separate column lists** are inferred for each input port — so a join's left-key dropdown shows only left-table columns and the right-key dropdown shows only right-table columns.
- Column inference works even before a flow has been run, by reading CSV headers or previously cached execution results.

Execution Engine

- **Topological sort** (Kahn's algorithm) ensures nodes run in dependency order. A cycle produces a clear error, not a hang.
 - **Port-aware routing**: edges connect named output ports to named input ports, so multi-output nodes (Merge/Join, Unique/Duplicate) can route each output stream independently.
 - **Node disabling**: any node can be toggled disabled. Disabled nodes are skipped; downstream nodes receive input from the last enabled upstream node.
 - **Per-node results**: after execution, each node stores its output DataFrame(s). The properties panel shows row/column counts per port and a live-updated code preview.
-

Python Code Export

The "Export Python" function generates a standalone `.py` file that reproduces the pipeline. Features of the generated code:

- Each node becomes one or more `pandas` expressions with a descriptive variable name derived from the tool type and node ID.
- Only the connected output ports of multi-output nodes are emitted — unused branches are not generated.
- The export respects disabled nodes (they appear as comments).
- The output file has no dependency on PyDataFlow — it only requires `pandas` (and `openpyxl` for Excel operations).

Example snippet for a join:

```
# --- Merge / Join (a3f8c1b2) ---
_m = pd.merge(df_import_csv_001, df_import_csv_002,
left_on='id', right_on=['customer_id'], how='outer',
indicator=True)
result_joined =
_m[_m['_merge'].isin(['both'])].drop(columns=['_merge']).reset_in
dex(drop=True)
result_left_unmatched =
_m[_m['_merge']=='left_only'].drop(columns=['_merge']).reset_index
(drop=True)
```

Project Persistence

Projects are saved as JSON with a version field. The file stores: - Node list: id, type, canvas position (x/y), configured parameters, disabled flag, annotation text - Edge list: source node + port → destination node + port

The format is human-readable and diffable in version control.

Technical Stack

Component	Technology
GUI framework	Python <code>tkinter</code> (stdlib) + <code>ttk</code> themed widgets
Data engine	<code>pandas</code>
Excel I/O	<code>openpyxl</code>
Canvas rendering	<code>tkinter.Canvas</code> with manual bezier curve drawing

Component	Technology
Layout	Three-pane (palette / canvas / properties) + bottom notebook tabs
Dependencies	Python 3.10+, pandas , openpyxl — nothing else
Platform	Cross-platform desktop (Windows, macOS, Linux)
Distribution	Run from source; no installer required

What It Is Not (Current Scope)

- **Not a BI / visualisation tool** — no charts, no dashboards. Output is tabular data or Python scripts.
- **Not a SQL tool** — all operations are expressed in pandas terms, not SQL.
- **Not a scheduling tool** — pipelines are run interactively, not on a schedule.
- **Not collaborative** — single-user, single-machine. No version control integration, no sharing.
- **Not a streaming tool** — batch processing of files only.
- **No database connectors** — input sources are CSV and Excel files only (currently).

Likely Comparison Points

When evaluating PyDataFlow against existing tools, the most relevant comparators are:

Tool	Overlap
Alteryx Designer	Closest conceptual match — node-based visual pipeline, local execution, code export
Tableau Prep	Visual pipeline builder focused on data shaping before Tableau dashboards
KNIME Analytics Platform	Open-source node-based pipeline; broader node library, Java-based
Microsoft Power Query (Excel/Power BI)	Visual M-language pipeline builder embedded in Microsoft products
Pentaho Data Integration (Kettle)	Enterprise ETL, node-based, Java

Tool	Overlap
Pandas itself / Jupyter notebooks	Code-first equivalent — PyDataFlow is essentially a visual front-end for pandas

PyDataFlow's distinguishing characteristics: fully local/offline, zero licensing cost, generates readable Python output, no Java runtime or electron bundle — just Python stdlib + pandas.